

ESTUDO APLICADO DE PROTOCOLOS COM APLICABILIDADE NO PROJETO INTEGRADOR

Projeto: Garimpo - Marketplace de Brechó

O Projeto Integrador desenvolvido pelo meu grupo, o Garimpo, é um marketplace de brechó que opera como uma aplicação web cliente-servidor: o navegador do usuário se comunica com um servidor remoto para trocar dados. Toda essa comunicação passa por uma pilha de protocolos de rede, e este relato descreve como cada um deles apareceu de forma concreta durante o desenvolvimento e os testes do projeto.

O protocolo TCP (Transmission Control Protocol) é o que garante que a comunicação entre o navegador e o servidor do Garimpo seja confiável. Antes de qualquer dado ser trocado, o TCP realiza um processo de três etapas para estabelecer a conexão, conhecido como three-way handshake: o cliente envia um segmento SYN indicando que quer se conectar, o servidor responde com SYN-ACK confirmando que está disponível, e o cliente finaliza com um ACK. Só então os dados começam a fluir. Durante o desenvolvimento, isso ficou evidente de duas formas. Primeiro, na aba Network das ferramentas de desenvolvedor do navegador, é possível ver o tempo que cada requisição leva para ser concluída decomposto em fases, e a primeira delas, chamada de Stalled ou Connecting, corresponde exatamente ao tempo gasto nesse handshake. Segundo, quando o servidor não estava em execução, o erro retornado pelo navegador era `ERR_CONNECTION_REFUSED`, que ocorre justamente quando o cliente tenta iniciar o handshake TCP e não recebe resposta, pois não há nenhum processo escutando na porta destino. Esse erro tornou visível algo que em teoria parece abstrato: sem o estabelecimento da conexão TCP, nenhum dado chega ao destino.

O TCP também é responsável por garantir que os dados chegam íntegros e na ordem correta. Cada segmento enviado carrega um número de sequência, e o receptor confirma o recebimento com um ACK. Se um segmento se perder no caminho, o TCP detecta a ausência de confirmação e o retransmite automaticamente. No contexto do Garimpo, isso significou que respostas longas do servidor, como a listagem completa de produtos, chegaram sempre completas ao navegador, mesmo que internamente tivessem sido divididas em vários segmentos pelo protocolo.

O HTTP (Hypertext Transfer Protocol) opera sobre essa conexão TCP já estabelecida. Ele define o formato das mensagens trocadas entre o navegador e o servidor: cada requisição é composta por uma linha inicial com o método e o caminho, seguida de cabeçalhos com metadados e, quando necessário, um corpo com dados. A resposta do servidor segue formato semelhante, com uma linha de status, cabeçalhos e corpo. O que o projeto evidenciou de forma prática sobre o HTTP não foi tanto os métodos ou os códigos de status, mas sim a natureza stateless do protocolo: o HTTP não mantém estado entre uma requisição e outra. Cada mensagem é independente e não tem memória da anterior. Isso ficou claro quando precisamos implementar autenticação: como o protocolo em si não guarda informação de sessão, foi necessário incluir manualmente em cada requisição um token de identificação do usuário, transmitido dentro de um dos cabeçalhos da mensagem. Sem essa informação explícita em cada troca, o servidor simplesmente não saberia quem estava fazendo a requisição.

O HTTPS é o HTTP com uma camada de criptografia adicionada pelo protocolo TLS (Transport Layer Security), que atua entre o TCP e o HTTP. Antes de qualquer mensagem HTTP ser transmitida, o TLS realiza seu próprio processo de handshake: as duas partes negociam qual algoritmo de criptografia usar e trocam chaves. A partir daí, todo o conteúdo

das mensagens é cifrado antes de ser entregue ao TCP para envio. No projeto, isso se tornou relevante no momento do deploy: ao migrar o servidor para a nuvem, a URL passou a usar HTTPS em vez de HTTP. Isso significa que qualquer dado trafegado, incluindo as credenciais de login dos usuários e os tokens de autenticação, passou a ser transmitido de forma criptografada, ilegível para qualquer intermediário que eventualmente intercepte os pacotes na rede.

O DNS (Domain Name System) é o protocolo responsável por traduzir nomes de domínio em endereços IP. Ele atua antes mesmo do TCP: para que o navegador consiga iniciar o handshake TCP com o servidor, ele precisa saber o endereço IP de destino. Se o usuário digitou um domínio, o sistema operacional consulta um servidor DNS que devolve o endereço IP correspondente. Durante o desenvolvimento local, o DNS não entrou em ação porque o endereço usado era localhost, que é resolvido diretamente pelo próprio sistema operacional sem consulta externa. Mas ao colocar o servidor em produção com um domínio público, o DNS passou a ser uma etapa real e obrigatória de cada conexão. Isso ficou evidente em alguns erros que ocorreram logo após o deploy e que inicialmente pareciam falhas no código, mas na verdade eram problemas na resolução do domínio: o navegador não conseguia obter o endereço IP do servidor, então nem chegava a tentar o handshake TCP.

Olhando para o projeto como um todo, ficou claro que esses protocolos formam uma pilha onde cada camada depende da anterior. O DNS entrega um endereço IP. O TCP usa esse endereço para estabelecer uma conexão confiável. O TLS, quando presente, adiciona criptografia sobre essa conexão. E o HTTP, no topo, define o formato das mensagens que carregam os dados da aplicação. Qualquer falha em uma dessas camadas interrompe tudo o que está acima dela, e foi exatamente isso que observamos em diferentes momentos do desenvolvimento: erros que apareciam na superfície como problemas da aplicação, mas que na verdade tinham origem em uma camada de protocolo mais baixa.